

《单片机原理及应用》—— C8051F020定时器/计数器





4.1 定时器/计数器

•定时和计数功能最终都是通过**计数实现**的，若计数的
事件源是周期固定的脉冲，则可以实现**定时功能**，否则
只能实现**计数功能**。因此可以将定时和计数功能由一个
部件实现。

•实现定时和计数的方法一般有**软件、专用硬件电路和
可编程定时器/计数器**三种方法。

- ▶ 软件：只能定时，且占用CPU时间，降低了CPU的使用效率。
- ▶ 专用硬件电路：可实现精确的定时和计数，但参数调节不便。
- ▶ 可编程定时器 / 计数器：不占用CPU时间，能与CPU并行工作，实现精确的定时和计数，又可以通过编程设置其工作方式和其它参数，因此使用方便。



C8051F020单片机定时器/计数器资源

有**5**个定时器/计数器：

其中3个（T0~T2）16位定时器/计数器，与标准8051中的计数器/定时器兼容

T3、T4为16位自动重装初值功能定时器/计数器，既可作为通用定时器使用，又可用于ADC、SMBus、UART。

T0/T1几乎完全相同，4种工作方式

T2/T4可分别作为UART0和UART1的波特率发生器



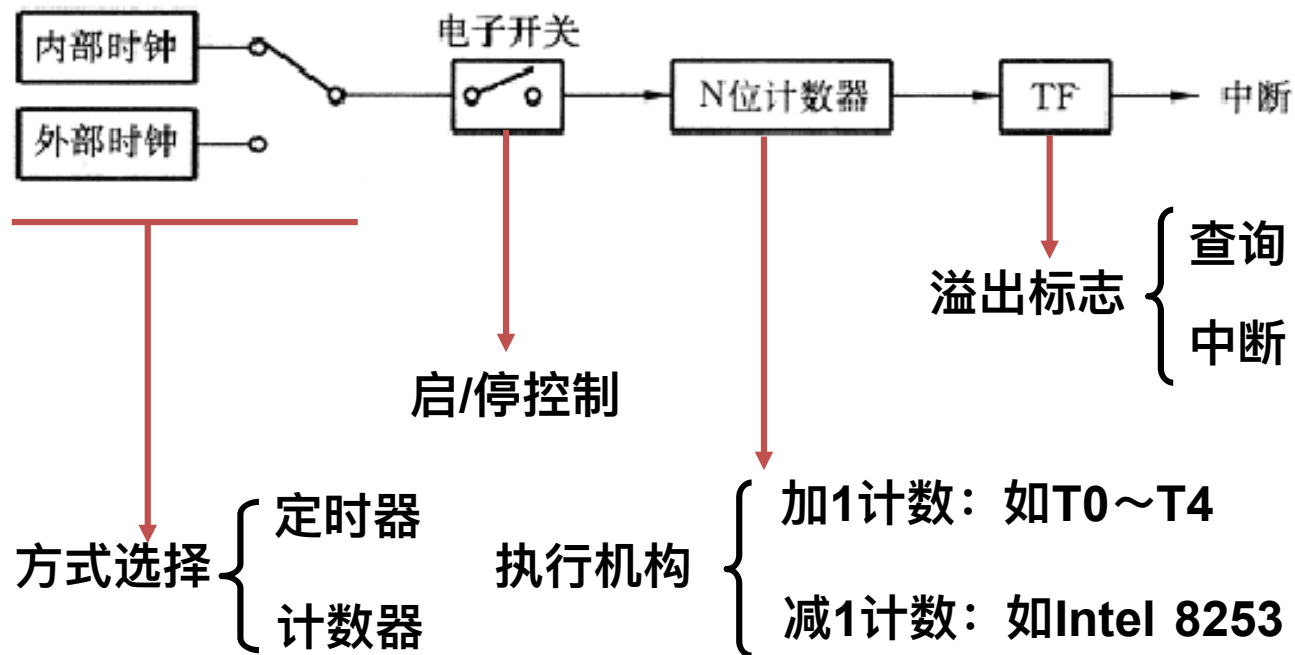
定时器/计数器工作方式

	T0/T1	T2/T4	T3
方式0	13位定时器/计数器	自动重载的16位定时器/计数器	自动重载的16位定时器/计数器
方式1	16位定时器/计数器	带捕捉的16位定时器/计数器	
方式2	8位自动重载的定时器/计数器	UART0/UART1的波特率发生器	
方式3	两个8位定时器/计数器（只限于定时器0）		



4.2 定时器的一般结构和工作原理

定时器/计数器由N位计数器、计数时钟源选择电路、计数控制开关和溢出状态标志等4部分组成



4.2.1 定时器模式

每一个计数周期($T_{\text{计数}}$)计数器加1, 直至**计满溢出**(从全1到全0)产生中断请求。对于一个N位的加1计数器, 若 $T_{\text{计数}}$ 是已知的, 则从初值a开始加1计数至溢出所占用的时间为:

$$T = T_{\text{计数}} \times (2^n - a)$$

最大定时时间

$$T_{\text{MAX}} = 2^N \times T_{\text{计数}}$$

式中**N由工作方式决定**, $T_{\text{计数}}$ 为定时器/计数器的**计数脉冲周期时间**, 由C8051F的主脉冲或主脉冲经12分频提供, 是否需要12分频取决于对时钟控制寄存器**CKCON**的设定 (提供12分频选项是为了与标准8051兼容)。

•T0~T4均为加1计数器



时钟控制寄存器CKCON

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
-	T4M	T2M	T1M	T0M	保留	保留	保留	00000000
位 7	位 6	位 5	位 4	位 3	位 2	位 1	位 0	SFR 地址: 0x8E

•位7：未用。读=0b，写=忽略。

•位6-3：T4M-T0M：T4到T0的时钟选择（不包含T3，T3的时钟选择由T3控制寄存器TMR3CN的第0位T3XCLK决定）。

0：定时器按系统时钟的12分频计数

1：定时器按系统时钟频率计数

•位2-0：保留。读=000b，写入值必须是000b。



外部输入信号的**下降沿触发计数**，计数器在每个时钟周期或时钟周期的12分频采样外部输入信号，若一个周期的采样值为1，下一个周期的采样值为0，则计数器加1，故识别一个从1到0的跳变需2个周期，所以，对外部输入信号最高的计数速率是时钟频率的 $1/2$ 或 $1/24$ （取决于是否12分频）。同时，外部输入信号的高电平与低电平保持时间均需大于一个周期。

如果从初始值TC开始计数，至计满溢出时共可计数 $2^N - TC$ 次。
最大可计数次数： 2^N 次

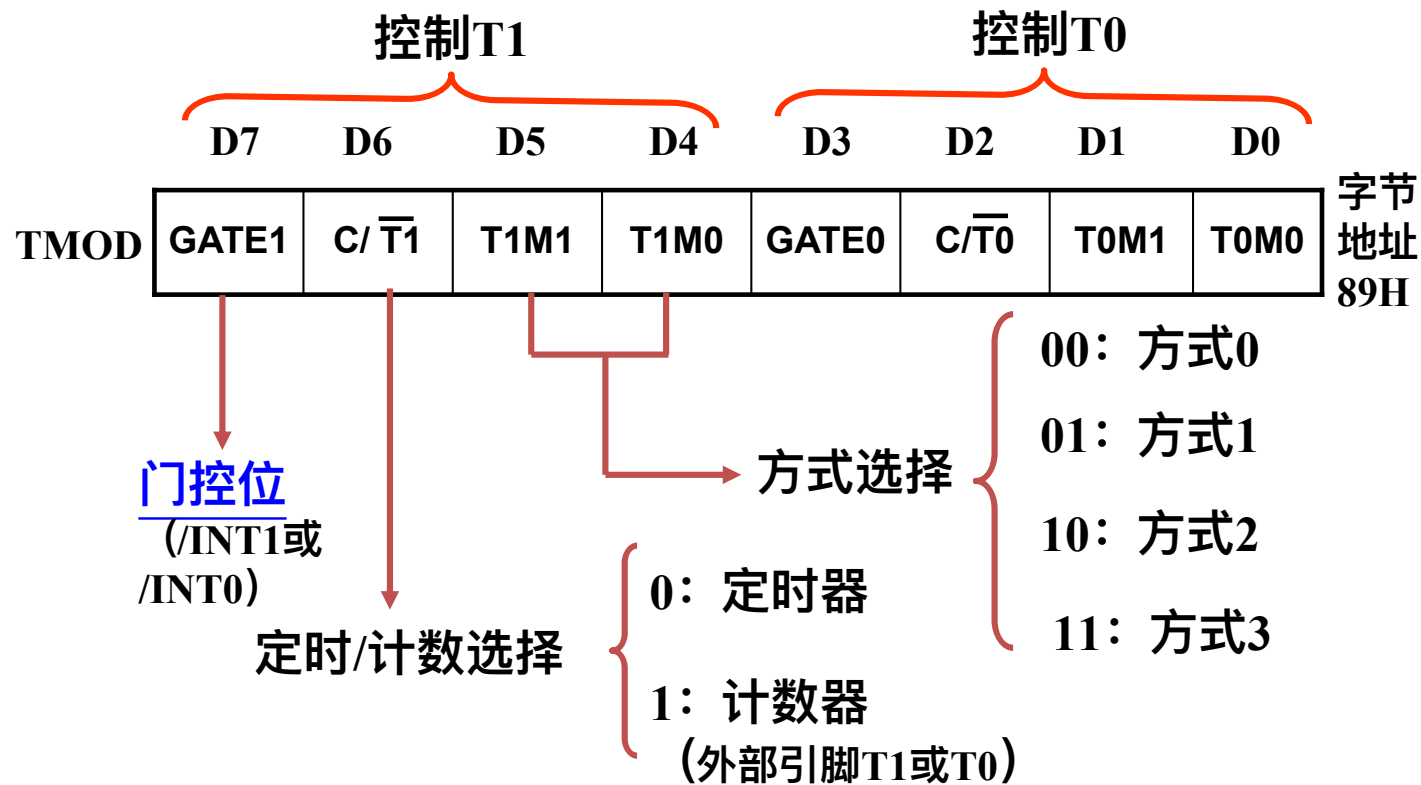
定时器/计数器一旦设置成某种方式并启动计数后，就会按设定的工作方式独立运行，不再占用CPU的操作时间，直到加1计数器计满溢出，才向CPU申请中断。



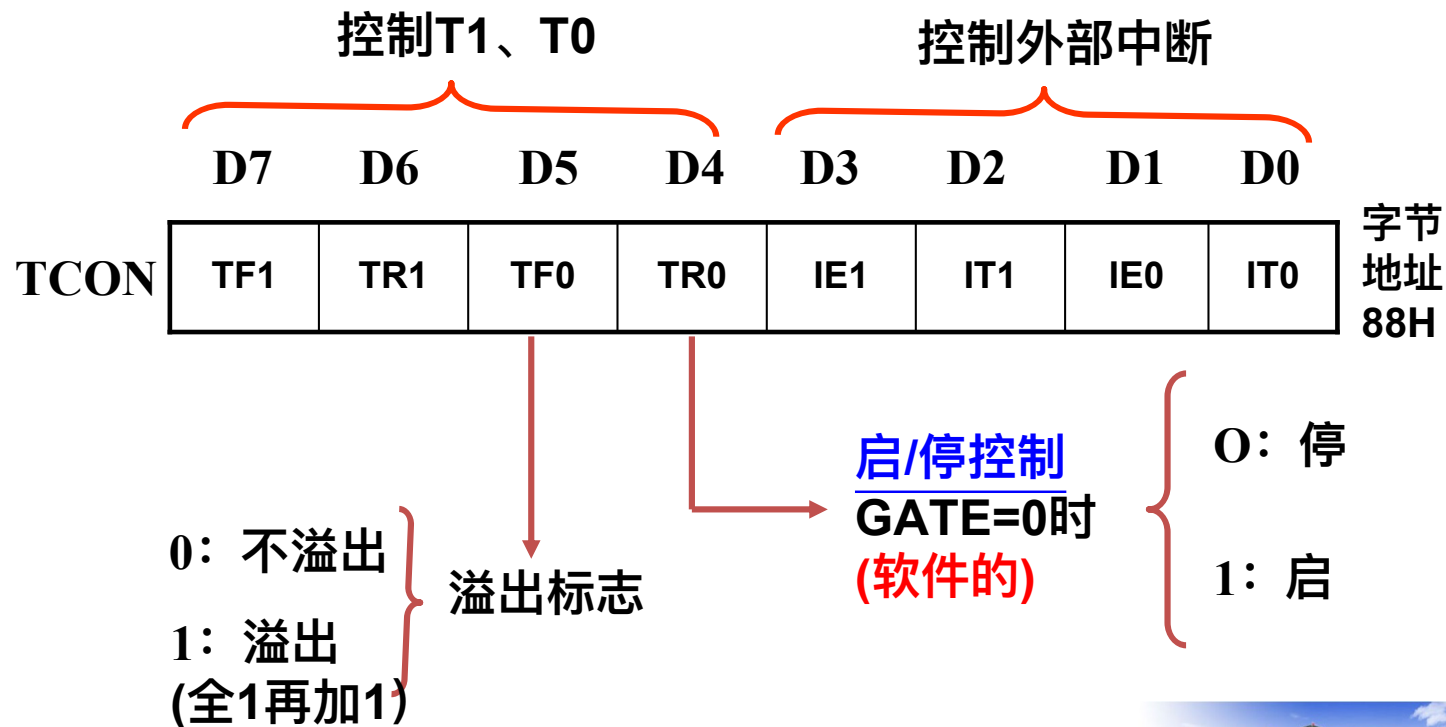
- 对定时器/计数器T0和T1的访问和控制是通过操作SFR实现的。
- T0和T1都是**16位的加1计数器**，访问时以两个字节的
形式出现：**TL0+TH0、TL1+TH1**
- TCON**用于允许/禁止定时器0和定时器1并指示它
们的工作状态。
- T0和T1都有**四种工作方式**，可以**TMOD**中的方式
选择位M1-M0进行选择。



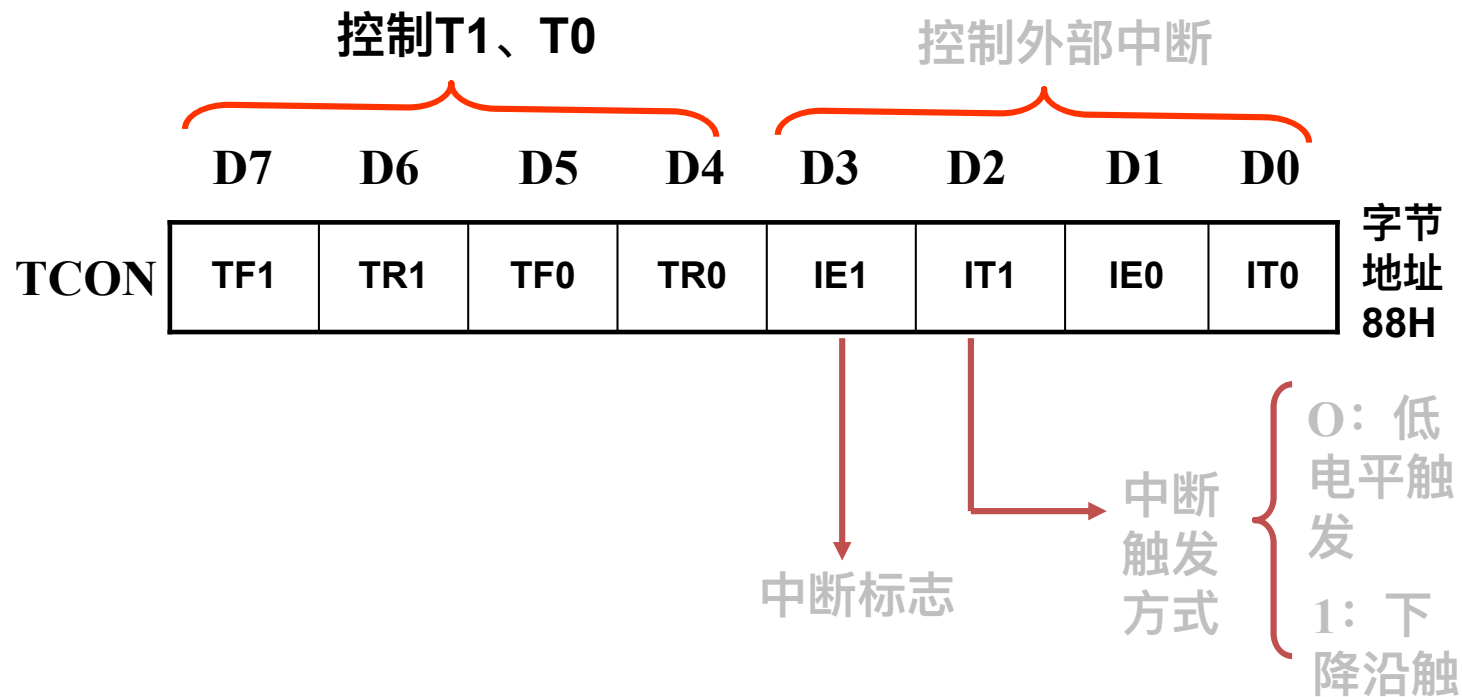
1、方式寄存器TMOD



2、控制寄存器TCON (1/2)

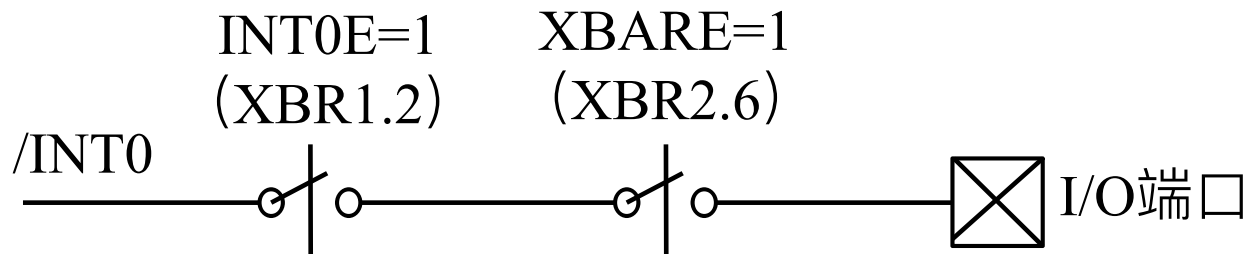
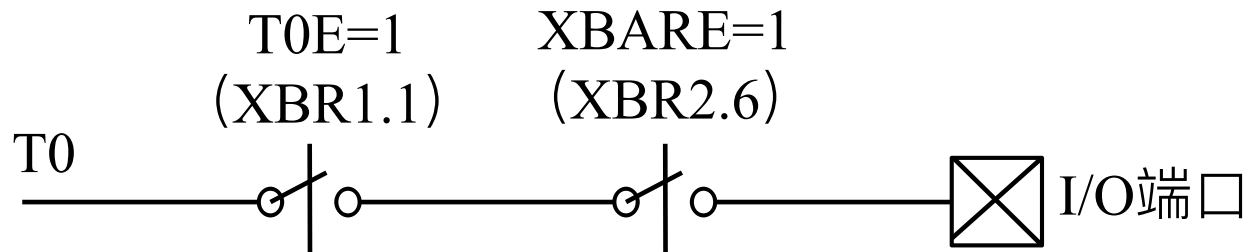


2、控制寄存器TCON (2/2)



3. T0和T1的交叉开关配置

对于T0，若要使用引脚T0和/INT0（计数方式）



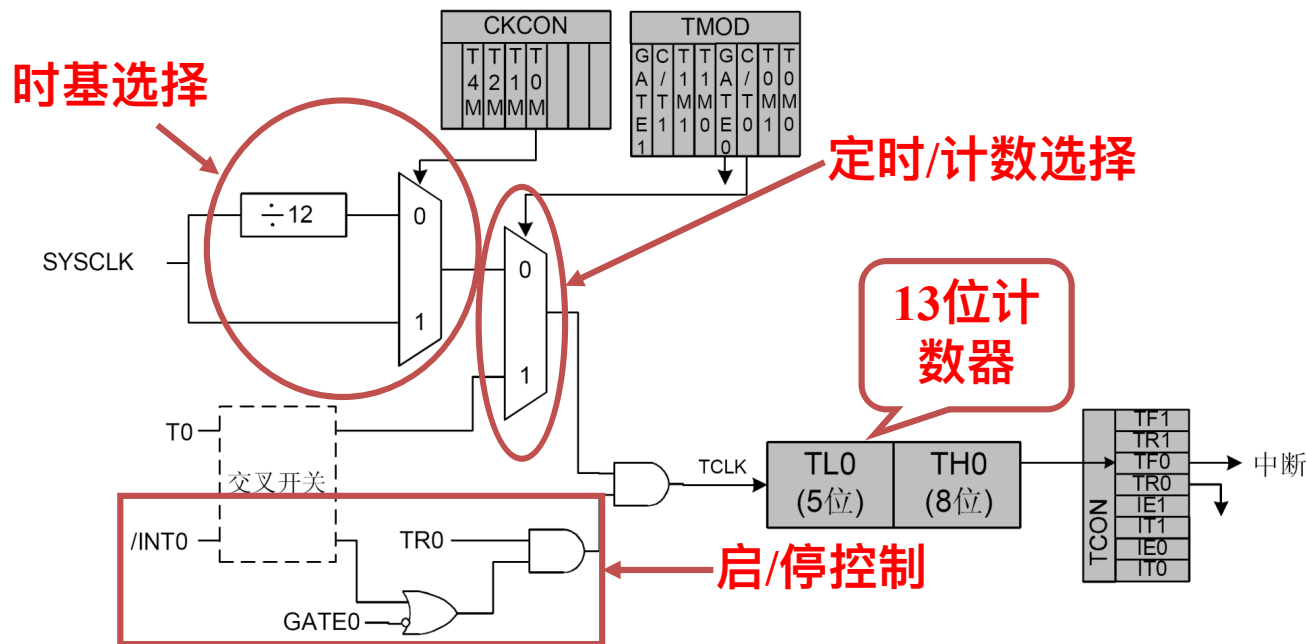
4. T0和T1的工作方式和计数器结构

表4-2 定时器T0、T1的工作方式

M1	M0	工作方式	功 能 说 明
0	0	0	13 位定时器 / 计数器
0	1	1	16 位定时器 / 计数器
1	0	2	自动重装初值的 8 位定时器 / 计数器
1	1	3	仅适用于 T0，分为两个 8 位计数器，T1 停止计数



(1) 工作方式0



GATE0=0 **TR0=** ———→ 启动计数

时:

1

GATE0=1 **TR0=1** 且 ———→ 启动计数

时:

INT0=1



(1) 工作方式0

•若T0工作于方式0的定时器模式，计数初值为a，
则T0从初值a加1计数至**溢出所需的时间为：**

$$T = \frac{12^{(1-T0M)}}{f_{osc}} \times (2^{13} - a) \mu s$$

式中 f_{osc} 为系统时钟频率，T0M为T0的时钟选择位。

❖ 如果 $f_{osc} = 12\text{MHz}$ ，则T0M=0时， $T = (2^{13} - a)\mu s$ ；
T0M=1时， $T = (2^{13} - a)/12\mu s$ 。

+ 2个周期



(2) 工作方式1

- 和方式0的差别仅仅在于计数器的位数不同，方式1为**16位**的定时器 / 计数器。
- T0工作于方式1时，由**TH0作为高8位**，**TL0作为低8位**，构成一个**16位计数器**。
- 若T0工作于方式1定时，计数初值为a， $f_{osc} = 12\text{MHz}$ ，则T0从计数初值a加1计数到溢出的定时时间为：

溢出标志要隔多久清除

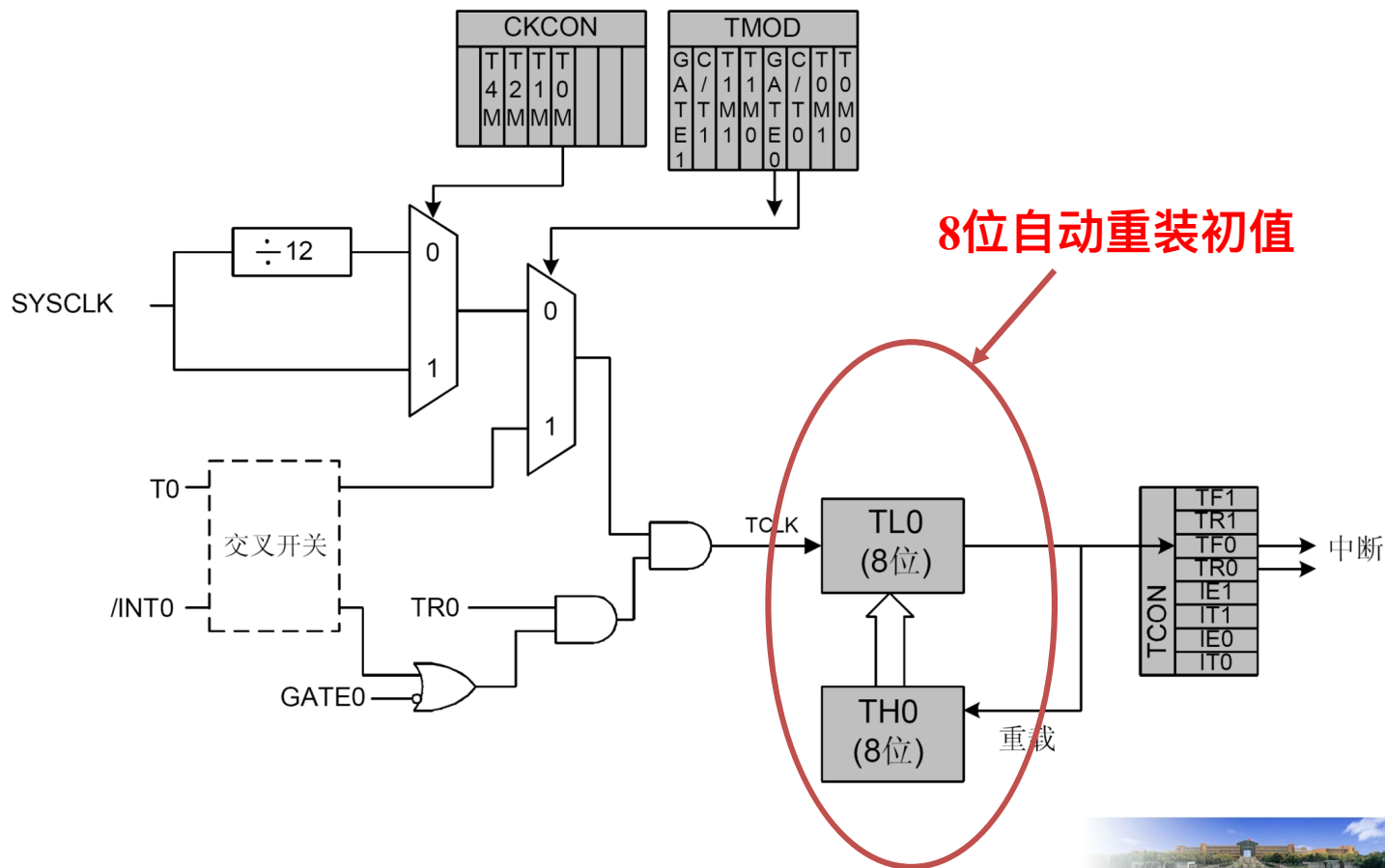
$$T = (2^{16} - a)\mu\text{s} \quad \text{或} \quad T = (2^{16} - a)/12\mu\text{s}.$$

$$2^{16} - 20000 = 65536 - 20000 = 45536$$

$$45536$$



(3) 工作方式2



(3) 工作方式2

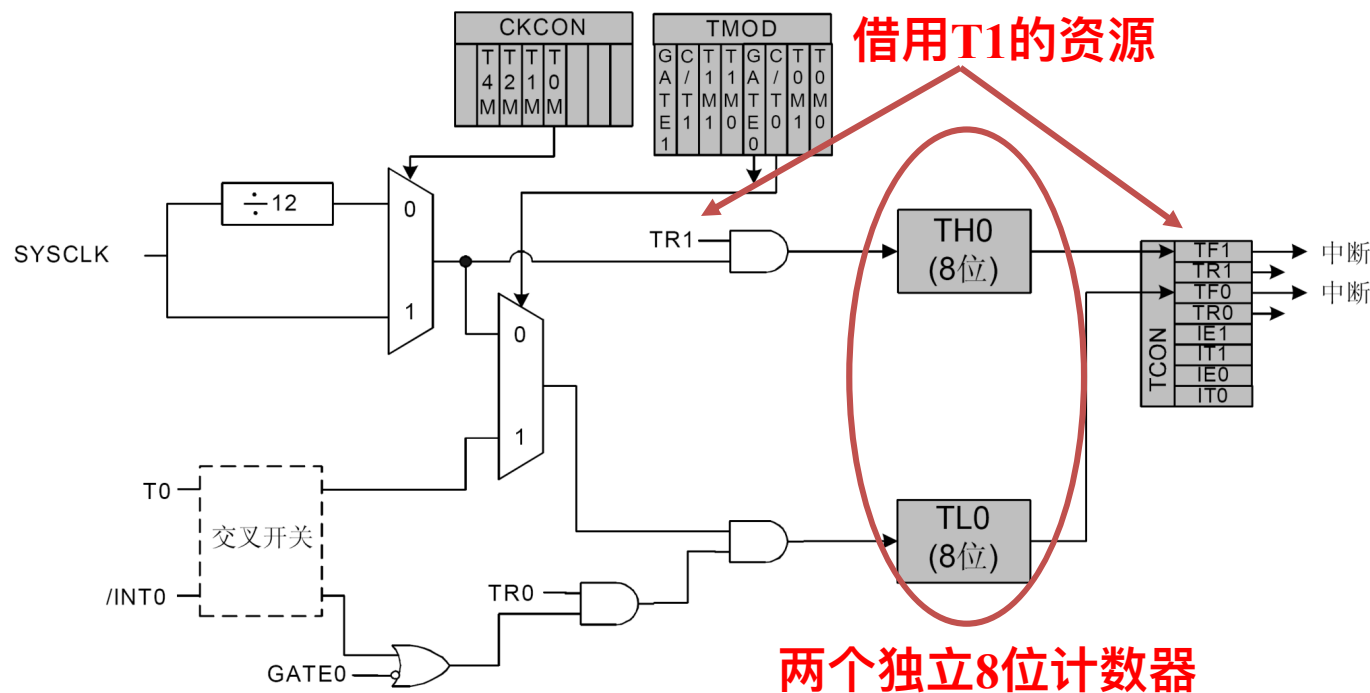
- 适用于需要重复定时或计数的场合。
- 定时精度比较高，但定时时间较短。
- 定时时间可用下式计算：

$$T = \frac{12^{(1-TO M)}}{f_{osc}} * (2^8 - a)$$



(4) 工作方式3

- 只适用于T0，若T1设置为方式3，则停止计数。



5. T0和T1的初始化

•初始化步骤

- 初始化TMOD
- 根据需要初始化CKCON
- 装入初值 (TH0、TL0)
- 中断设置 (IE、IP)
- 启动定时/计数器 (TCON)



• 计数器方式初值的计算

$$TC = M - C$$

M为计数器的模，与工作方式有关，C为需要的计数值

• 定时器方式初值的计算

$$T = (M - TC) \times T_{\text{计数}} \quad T_{\text{计数}} = T_{\text{CLK}} \text{ 或 } 12T_{\text{CLK}}$$

$$TC = M - T / T_{\text{计数}} = 2^N - \frac{T}{12^{1-T0M}} \times f_{\text{osc}}$$

• 最大定时时间 ($f_{\text{OSC}} = 12\text{MHz}$ 、 $T0M=0$) :

— 方式0: $T_{\text{MAX}} = 2^{13} \times 1\mu\text{s} = 8.192\text{ms}$

— 方式1: $T_{\text{MAX}} = 2^{16} \times 1\mu\text{s} = 65.536\text{ms}$

— 方式2、3: $T_{\text{MAX}} = 2^8 \times 1\mu\text{s} = 0.256\text{ms}$



6. T0和T1的应用举例

•**例1** 若 $f_{OSC}=12\text{MHz}$ ，用系统时钟的十二分频作为计数源，请计算定时 2ms 所需的初值，并给出初始化程序。

•**解：** $\because f_{OSC}=12\text{MHz}$ ，用系统时钟的十二分频作为计数源时，方式2、3的最大定时时间只有 0.256ms ，因此要想获得 2ms 的定时时间，**采用方式0或方式1。**

•方式0

- $TC=2^{13}-2\text{ms}/1\mu\text{s}=6192=1830\text{H}=0001100000110000\text{B}$
- 即：TH0=0C1H；TL0=10H（高三位为0）

•方式1

- $TC=2^{16}-2\text{ms}/1\mu\text{s}=63536=\text{F830H}$
- 即：TH0=0F8H；TL0=30H



- 初始化TMOD
- 根据需要初始化CKCON
- 装入初值 (TH0、TL0)
- 中断设置 (IE、IP)
- 启动定时/计数器 (TCON)



► 初始化TMOD TMOD=0x01; //T0, 方式1

位7	位6	位5	位4	位3	位2	位1	位0	复位值
GATE1	C/ $\overline{T1}$	T1M1	T1M0	GATE	C/ $\overline{T0}$	T0M1	T0M0	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0x89

0 0 0 0 0 0 0 1

门控位

定时/计数选择 {
0: 定时器
1: 计数器

方式选择

{
00: 方式0
01: 方式1
10: 方式2
11: 方式3



► 初始化CKCON

CKCON &= 0xF7;

//12分频

位7	位6	位5	位4	位3	位2	位1	位0	复位值
-	T4M	T2M	T1M	T0M	保留	保留	保留	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0x8E

0

•位6-3: T4M-T0M: T4到T0的时钟选择 (不包含T3, T3的时钟选择由T3控制寄存器TMR3CN的第0位T3XCLK决定)。

0: 定时器按系统时钟的12分频计数

1: 定时器按系统时钟频率计数



► 装入初值

$$TC = 2^{16} - 2\text{ms}/1\mu\text{s} = 63536 = \text{F830H}$$

即: $\text{TH0} = \text{F8H}$; $\text{TL0} = 30\text{H}$

$\text{TH0} = 0\text{xF8};$

$\text{TL0} = 0\text{x30};$



► 中断设置 (IE、IP)

• **IE** $IE = 0x02;$ //允许T0中断

位7	位6	位5	位4	位3	位2	位1	位0	复位值
EA	IEGF0	ET2	ES0	ET1	EX1	ET0	EX0	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xA8

• **IP** $IP = 0x02;$ //T0中断为高优先级

位7	位6	位5	位4	位3	位2	位1	位0	复位值
-	-	PT2	PS0	PT1	PX1	PT0	PX0	11000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xB8



•初始化程序

```
void T0_mode1_2ms_init()
```

```
{
```

```
    WDT_Init();           //看门狗初始化
```

```
    TMOD = 0x01;          //T0, 方式1
```

```
    CKCON &= 0xF7;        //T0计数源选择系统脉冲的12分频
```

```
    TH0 = 0xF8;           //初值
```

```
    TL0 = 0x30;
```

```
    IE |= 0x02;           //允许T0中断, 可用ET0=1代替
```

```
    IP |= 0x02;           //T0中断为高优先级, 可用PT0=1代替
```

```
    TCON|=0x10;          //启动T0, 可用TR0=1代替
```

```
}
```



•给定时器赋初值的语句也可以采用如下方法：

$TH0 = (65536 - 2000) / 256;$

$TL0 = (65536 - 2000) \% 256;$

或

$TH0 = -2000 / 256;$

$TL0 = -2000 \% 256;$



• **例4.2** 若 $f_{osc}=12\text{MHz}$, T1工作于方式1, 产生50ms的定时中断, TF1为高级中断源。试编写主程序和中断服务程序, 使P1.0产生**周期为1s**的**方波**。

• **解:** 让P1.0每**500ms****取反一次**即可实现。定时器的单次定时时间不可能达到500ms, 可让定时器**多次定时产生500ms**的定时时间, 如让T1工作在方式1, 单次定时时间为50ms, 那么T1中断10次就是500ms的时间。

• (1) 确定定时常数

- 假设使用 f_{osc} 的12分频作为计数源, 则 $T_{计数} = 12 / f_{osc} = 12 / (12 \times 10^6) = 1\mu\text{s}$
- 由公式 $TC = M - T / T_{计数}$, 可知 $TC = 2^{16} - 50 \times 10^3 = 15536 = 3CB0H$
- $\therefore TH1=0x3c, TL0=0xb0。$



• (2) 初始化程序

包括T1初始化和中断系统初始化，主要是对IP、IE、CKCON、TCON、TMOD的相应位进行正确的设置，并将时间常数送入T1。本例中将初始化操作放在主程序中完成，当程序规模较大时，应编写单独的初始化程序，以利于程序的模块化设计。

• (3) 中断服务程序

中断服务程序除了完成要求的方波产生这一工作之外，还要注意将时间常数重新送入T1中，为下一次产生中断作准备。



6. T0和T1的应用举例 (3/6)

程序清单如下 (主程序) :

```
#include <c8051f020.h>
sbit P1_0 = P1^0;
int count=10;           //10次T1中断为500ms
void main( void )
{
    CKCON&=0xef;        //T1的计数源选择系统脉冲的12分频
    TMOD=0x10;           //T1方式1
    P1_0=0;
    TH1=0x3c;            //初值
    TL1=0xb0;
    IE|=0x88;            //允许T1中断
    IP|=0x08;            //TF1中断为高级中断
    TCON|=0x20;          //启动T1
    while(1);            //死循环，等待中断，产生方波
}
```



6. T0和T1的应用举例 (4/6)

程序清单如下 (中断服务程序) :

```
void Timer1_ISR (void) interrupt 3
{
    TH1=0x3c;           //重装初值
    TL1|=0xb0;          //进入中断的过程 计数器还在跑, 用
                        //中断计数 或可以保留多出的几个计数器
    count--;
    if (count==0)       //500ms到, 重赋计数初值, P1.0取反
    {
        count=10;  P1_0=!P1_0;
    }
}
```

问题: 为什么用TL1|=0xb0;而非TL1=0xb0?



6. T0和T1的应用举例 (5/6)

程序清单如下 (查询式程序) :

```
•#include <c8051f020.h>

sbit P1_0= P1^0;

void main( )
{
    int count=10;//10次T1中断为500ms

    CKCON&=0xef;    //T1的计数源选择系统脉冲的12分频
    TMOD=0x10;      //T1方式1
    P1_0=0;
    TR1=1;           //启动T1
```



6. T0和T1的应用举例 (6/6)

```
For(; ;)                //死循环，产生方波
{
    TH1=-50000/256;      //T1初值
    TL1=-50000%256;
    while(!TF1); //查询等待TF1置位，
    TF1=0;
    If (count!=0)  count--;
    else {
        count=10;
        P1_0=!P1_0;
    }
}
}
```



- T2为**16位**定时/计数器，由**TL2**（低字节）和**TH2**（高字节）组成。
- **C/T2=0(定时)**时，系统时钟作为定时器的输入（由CKCON的T2M位指定不分频或12分频）。**C/T2 =1（计数）**时，T2输入引脚上的负跳变使计数器加“1”。
- T2还可以用于启动ADC数据转换。

• **三种工作方式**（由T2CON中的配置位选择）：

- (1) 自动重装初值的16位定时器/计数器方式**
- (2) 带捕捉的16位定时器/计数器方式**
- (3) 波特率发生器方式**



1. T2控制寄存器T2CON

位7	位6	位5	位4	位3	位2	位1	位0	复位值
TF2	EXF2	RCLK 0	TCLK0	EXEN2	TR2	C/T2	CP/RL 2	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xC8

•位7 (TF2) : T2溢出标志位

- T2溢出时由硬件置位。允许T2中断时，使CPU转向T2的中断服务程序。不能由硬件自动清0，**必须用软件清0**。
- RCLK0或TCLK0为1时（波特率发生器方式），TF2不会被置1。

•位6 (EXF2) : T2外部中断标志位

- EXEN2为“1”时，当T2EX输入引脚发生**负跳变**时，由硬件置位。允许T2中断时，使CPU转向T2的中断服务程序。不能由硬件自动清0，**必须用软件清0**。



位7	位6	位5	位4	位3	位2	位1	位0	复位值
TF2	EXF2	RCLK ₀	TCLK0	EXEN2	TR2	C/T2	CP/RL ₂	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xC8

• **位5 (RCLK₀)** : UART0接收时钟选择标志位

– 0: T1溢出作为接收时钟。1: T2溢出作为接收时钟。

• **位4 (TCLK0)** : UART0发送时钟选择标志位

– 0: T1溢出作为发送时钟。1: T2溢出作为发送时钟。

• **位3 (EXEN2)** : T2外部中断允许标志位

– 0: T2EX上的负跳变被忽略。

– 1: T2EX上的负跳变导致一次捕捉或重载。

• **位2 (TR2)** : T2启/停控制位

– 0: 停止。1: 启动。



位7	位6	位5	位4	位3	位2	位1	位0	复位值
TF2	EXF2	RCLK 0	TCLK0	EXEN2	TR2	C/T2	CP/RL 2	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址： 0xC8

•位1 (C/T2)：定时器/计数器功能选择位

- 0：定时器功能，由T2M(CKCON.5)定义的时钟加“1”。
- 1：计数器功能，由外部输入引脚(T2)的负跳变加“1”。

•位0 (CP/RL2)：捕捉/重载选择位

- EXEN2必须为1才能使T2EX上的负跳变能够被识别并触发捕捉和重载。当RCLK0或TCLK0为“1”时，该位被忽略，T2将工作在自动重装载方式。
- 0：T2溢出或T2EX上发生负跳变时将自动重装载
- 1：T2EX发生负跳变时捕捉。



2. T2的工作方式和计数器结构

表 T2的方式选择

RCLK0+TCLK0	CP/RL2	工作方式
0	0	自动重装载的16位定时器/计数器
0	1	带捕捉的16位定时器/计数器
1	×	UART0波特率发生器方式

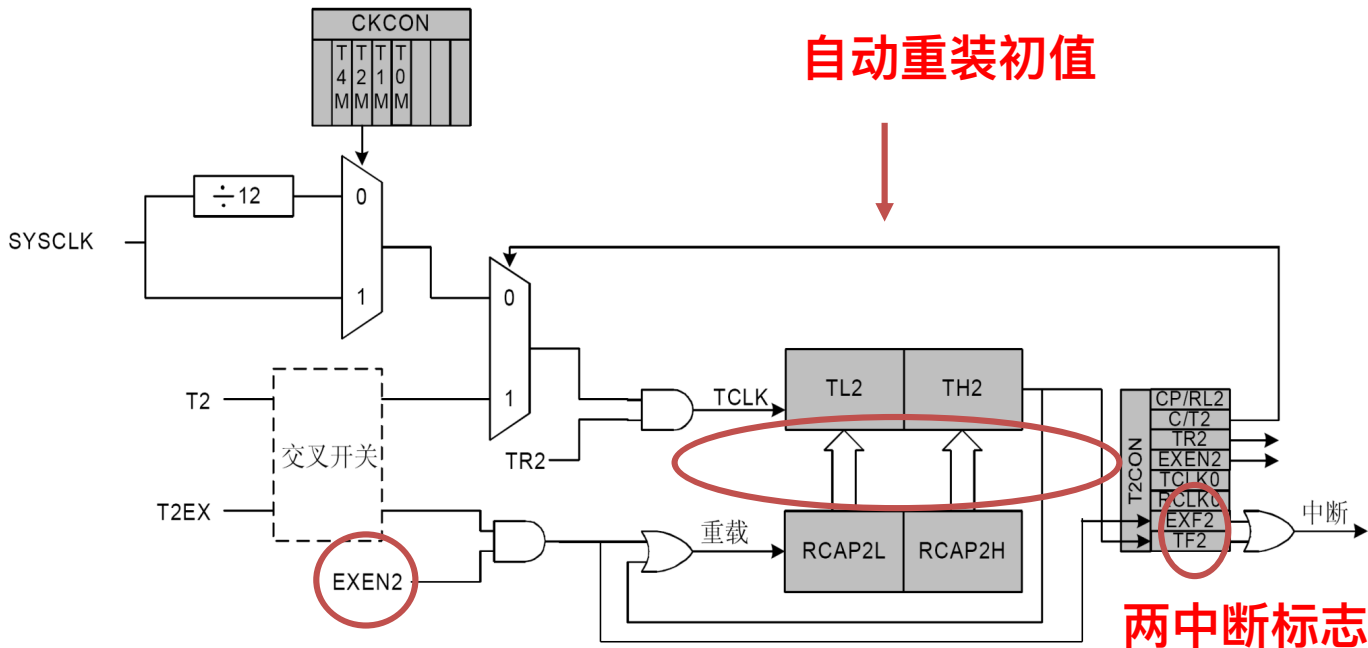
• (1) 方式0：自动重装初值的16位定时器/计数器方式

– 原理框图如所示。

– TH2、TL2构成16位加“1”计数器。RCAP2H、RCAP2L构成16位初值寄存器，EXEN2=1时，当**T2EX上有负跳变或T2溢出**时，将RCAP2H、RCAP2L中预置的初值自动重新装入TH2、TL2，T2重新开始计数，并置位中断标志EXF2或TF2，向CPU申请中断。EXEN2=0时，T2EX上的负跳变被忽略，只有当T2溢出时才重载初值并向CPU申请中断。



(1) 方式0：16位自动重装初值方式



**为0时忽略T2EX负跳变，
不产生EXF2中断**

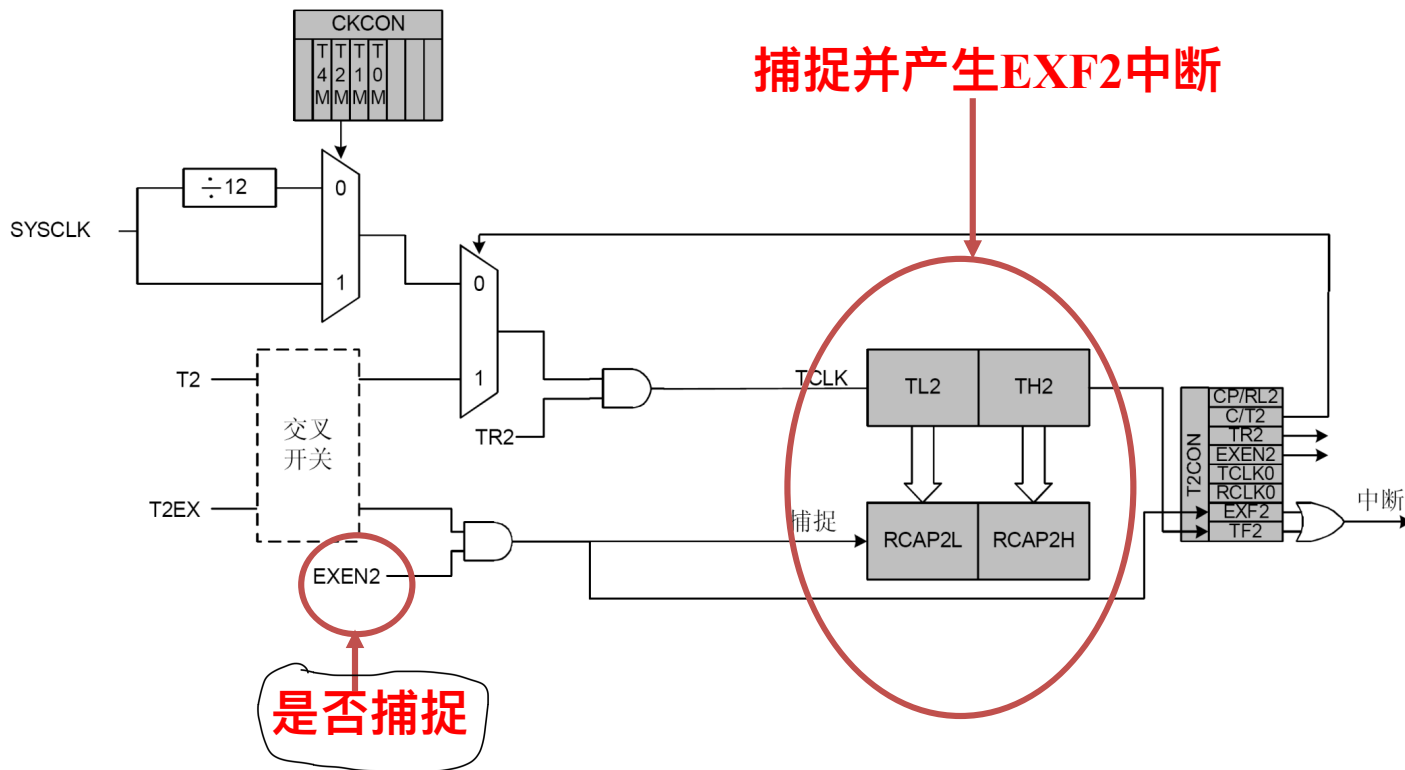


(2) 方式1: 16位带捕捉方式

- $RCLK0=0$ 、 $TCLK0=0$ 、 $CP/RL2=1$ 时, T2工作在此方式
- **$EXEN2=1$ 时为允许捕捉方式**, T2EX引脚上的负跳变将TH2、TL2的当前值捕捉到RCAP2H、RCAP2L寄存器, 同时置EXF2=1, 发出中断请求。
- $EXEN2=0$ 时, RCAP2H、PCAP2L不起作用, **此时T2与T1、T0的方式1完全相同**。即: $C/T2=0$ 时为16位定时器方式, $C/T2=1$ 时为16位计数器方式, 计数溢出时TF2=1, 发送中断请求信号。
- 原理框图如**图**所示。 会计数, 会溢出, 但不会捕捉



(2) 方式1：16位带捕捉方式



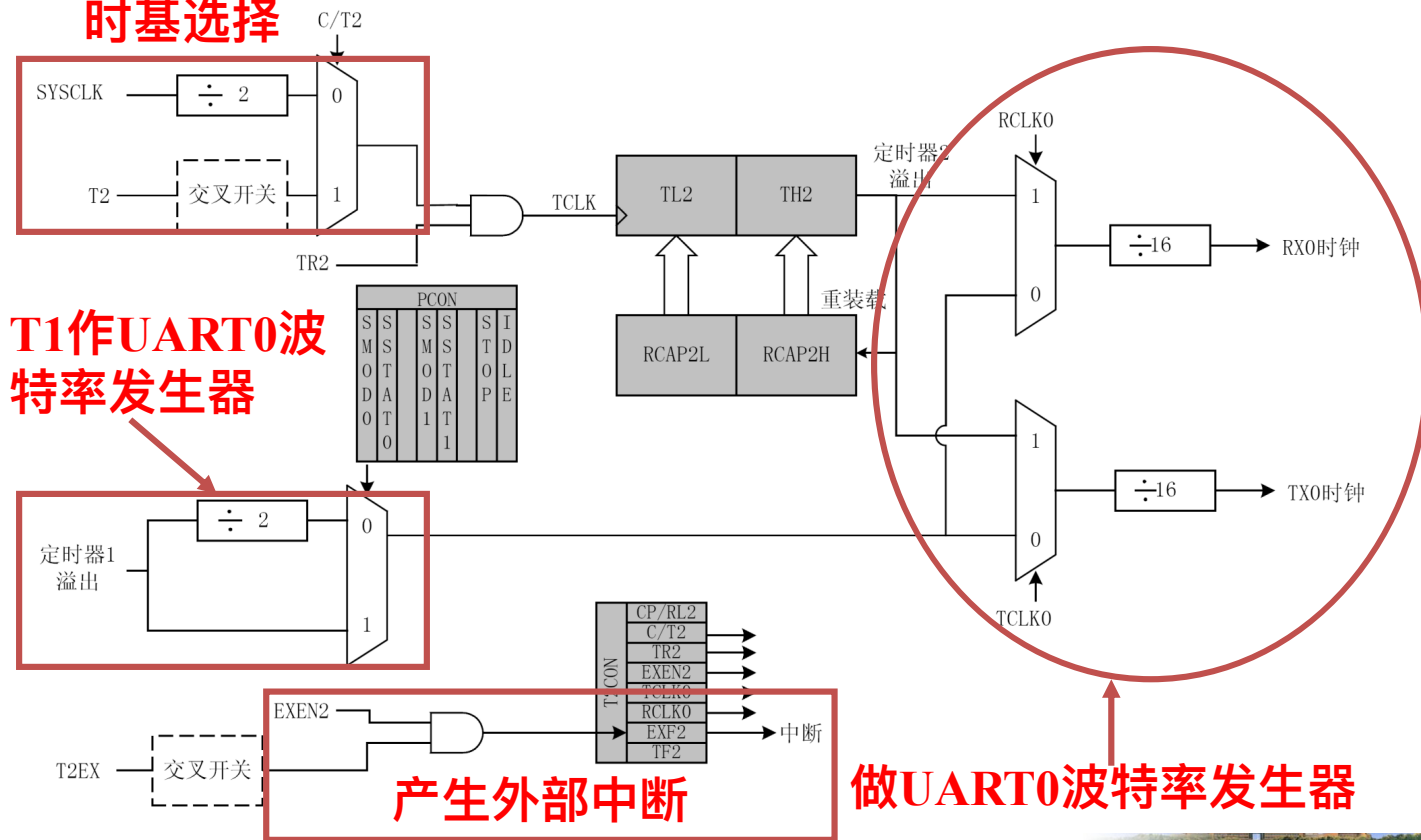
(3) 方式2: 波特率发生器方式

- RCLK或TCLK置1时, T2工作于波特率发生器方式。
- 与自动重装载方式相似。但不置位TF2, 也不产生中断。溢出事件用作UART0的移位时钟输入。
- T2溢出可用于产生独立的发送或接收波特率, 也可同时产生发送和接收波特率, 取决于T2CON的设置。
- T2的计数源可以是系统时钟的二分频, 也可以是T2引脚上的输入, 取决于C/T2的设置。
- 如果EXEN2为1, 则T2EX引脚上的负跳变将置位EXF2标志, 并产生一个T2中断 (如果允许)。因此, T2EX 输入可以被用作额外的外部中断源。
- 原理框图如图所示。



(3) 方式2：波特率发生器方式

时基选择



(3) 方式2：波特率发生器方式

- 当选择系统时钟的二分频做计数源时，T2 为UART0提供的波特率可以用如下公式计算：

$$\text{波特率} = \frac{\text{SYSCLK}}{32 \times (65536 - [RCAP2H: RCAP2L])}$$

一般是已知波特率，
求寄存器的初值。

系统时钟2分频，溢出信号16分频用作UART0的移位时钟脉冲

- 当选择外部引脚T2上的输入作为时基时，T2为UART0提供的波特率可以用如下公式计算：

$$\text{波特率} = \frac{F_{CLK}}{16 \times (65536 - [RCAP2H: RCAP2L])}$$



5 定时器/计数器T4

- 16位的定时器/计数器，由**TL4**（低字节）和**TH4**（高字节）组成。
- T4还可以用于启动ADC 数据转换。
- T4与T2的功能和结构类似，与T2有区别的是，**T4是作为UART1的波特率发生器**。

表4-4 定时器T4的方式选择

RCLK1+TCLK1	CP/RL4	工作方式
0	0	自动重载的16位定时器/计数器
0	1	带捕捉的16位定时器/计数器
1	×	UART1波特率发生器方式



T4CON的格式:

位7	位6	位5	位4	位3	位2	位1	位0	复位值
TF4	EXF4	RCLK 1	TCLK1	EXEN4	TR4	C/T4	CP/RL 4	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xC9

•位7 (TF4) : T4溢出标志位

- T4溢出时由硬件置位。允许T4中断时，使CPU转向T4的中断服务程序。不能由硬件自动清0，必须用软件清0。
- RCLK0或TCLK0为1时（波特率发生器方式），TF4不会被置1。

•位6 (EXF4) : T4外部中断标志位

- EXEN4为“1”时，当T4EX输入引脚发生负跳变时，由硬件置位。允许T4中断时，使CPU转向T4的中断服务程序。不能由硬件自动清0，必须用软件清0。



位7	位6	位5	位4	位3	位2	位1	位0	复位值
TF4	EXF4	RCLK 1	TCLK1	EXEN4	TR4	C/T4	CP/RL 4	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xC9

• 位5 (**RCLK1**) : UART1接收时钟选择标志位

- 0: T1溢出作为接收时钟。1: T4溢出作为接收时钟。

• 位4 (**TCLK1**) : UART1发送时钟选择标志位

- 0: T1溢出作为发送时钟。1: T4溢出作为发送时钟。

• 位3 (**EXEN4**) : T4外部允许标志位

- 0: T2EX上的负跳变被忽略。
- 1: T2EX上的负跳变导致一次捕捉或重载。

• 位2 (**TR4**) : T4运行控制位

- 0: 禁止T4。1: 允许T4。



位7	位6	位5	位4	位3	位2	位1	位0	复位值
TF4	EXF4	RCLK 1	TCLK1	EXEN4	TR4	C/T4	CP/RL 4	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xC9

• 位1 (C/T4)：定时器/计数器功能选择位

- 0：定时器功能：T4由T4M(CKCON.6)定义的时钟加“1”。
- 1：计数器功能：T4由外部输入引脚(T4)的负跳变加“1”。

• 位0 (CP/RL4)：捕捉/重载选择位

- 该位选择T4为捕捉还是自动重载方式。EXEN4必须为1才能使T4EX上的负跳变能够被识别并触发捕捉和重载。当RCLK0或TCLK0为“1”时，T4将工作在自动重载方式。
- 0：当T4溢出或T4EX上发生负跳变时将自动重载
- 1：在T4EX发生负跳变时捕捉。

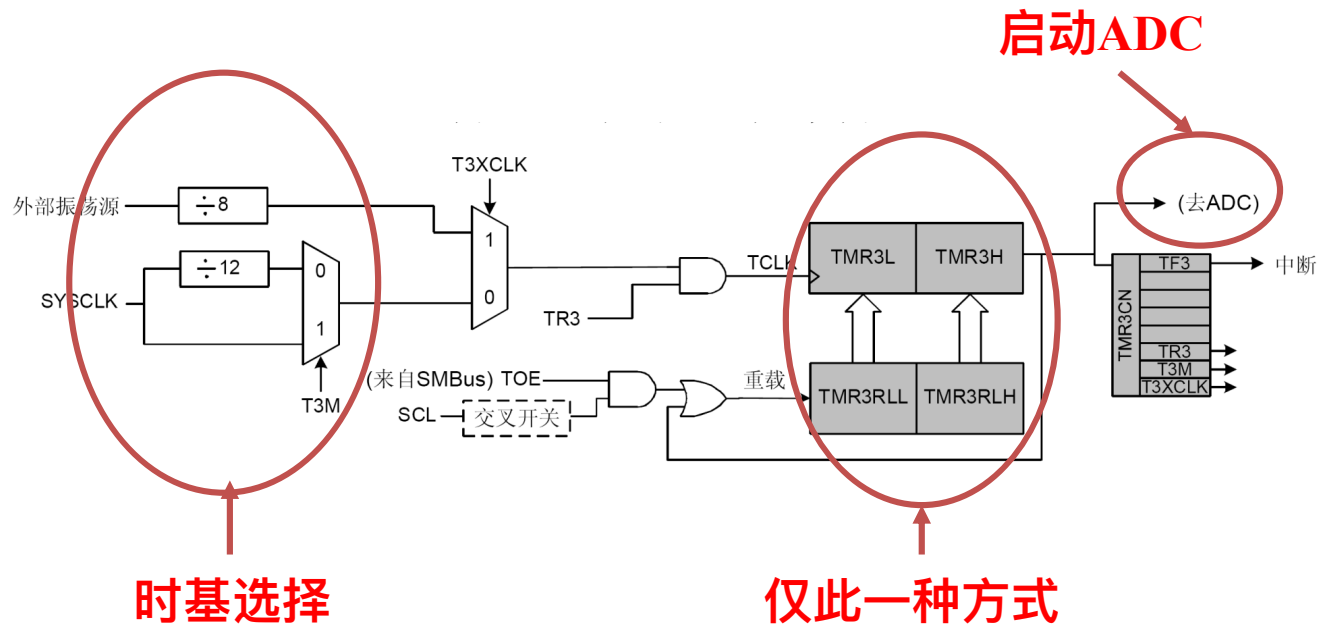


1. 定时器T3的结构

- 16位定时/计数器，由**TMR3L**（低字节）和**TMR3H**（高字节）组成
- T3的时钟输入可以通过程序选择为外部振荡器的8分频、系统时钟或系统时钟的12分频。
- T3只有自动重装初值一种工作方式，初值保存在**TMR3RL**（低字节）和**TMR3RH**（高字节）两个SFR中，T3没有计数器方式。
- 除作为通用定时/计数器使用外，T3还可以用于启动ADC数据转换、SMBus定时等。
- 原理框图如图4所示。



1. 定时器T3的结构



2. 定时器3控制寄存器

TMR3CN

位7	位6	位5	位4	位3	位2	位1	位0	复位值
TF3	-	-	-	-	TR3	T3M	T3XCLK	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0x91

• 位7 (TF3) : T3溢出标志位

– 溢出时置1，不能由硬件自动清0，必须用软件清0

• 位6-3：未用。读=0000b，写=忽略

• 位2 (TR3) : T3运行控制位

– 0：停止。1：启动。

• 位1 (T3M) : T3时钟选择位

– 0：T3使用系统时钟的12分频。1：T3使用系统时钟。

• 位0 (T3XCLK) : T3外部时钟选择位

– 0：由T3M定义。1：外部振荡器输入的8分频。



3. T3应用举例

• **例 4.3** 假设C8051F020的并行口P2、P3连接16个LED指示灯，试编写程序使P3口所接的LED灯循环点亮，P2口所接的LED灯实现走马灯效果，要求指示灯状态每秒刷新2次。（实验四）

• **解：**要实现题目要求的效果，只需要定期更新P2、P3口的状态即可。这里可以使用T3定时器再加软件计数的方法达到所要求的时间，假设T3定时0.1秒产生中断，则软件计数器每0.1秒加1，让计数器加到5时，改变P2、P3口的状态，就可以实现每秒2次刷新LED灯的状态。



3. T3应用举例

```
#include <c8051f020.h>
```

```
sfr16 TMR3RL = 0x92;           //16位SFR
```

```
sfr16 TMR3 = 0x94;
```

```
#define SYSCLK 2000000          //系统时钟使用2MHz
```

```
//函数声明
```

```
void PORT_Init(void);
```

```
void Timer3_Init(int counts);
```

```
void Timer3_ISR(void);
```

```
//P2口8个LED（共阳极）产生走马灯效果所需的数据
```

```
unsigned int xdata p2led[]={0x7F, 0xBF, 0xDF, 0xEF, 0xF7, 0xFB, 0xFD,  
0xFE};
```



3. T3应用举例

```
void main (void)
```

```
{
```

```
    WDTCN = 0xde;
```

//禁止看门狗定时器

```
    WDTCN = 0xad;
```

```
    PORT_Init();
```

//端口初始化

```
    Timer3_Init(SYSCLK/12/10);
```

//T3初始化, 产生0.1秒的定时中断

```
    EA = 1;
```

//开中断

```
    while (1);
```

//循环等待T3中断, 产生走马灯效果

```
}
```

```
void PORT_Init (void)
```

```
{
```

```
    XBR2 = 0x40;
```

//使能交叉开关

```
}
```

$$f = 1k.$$

$$T = 0.001$$

$$10^6$$

$$M$$

$$\frac{1}{1000} = 1 \mu s$$



3. T3应用举例

void Timer3_Init (int counts)

{ 需要将计数器的地址定义到 TMR3RL 寄存器.

TMR3RL = -counts;

TMR3 = 0xFFFF;

EIE2 |= 0x01;

TMR3CN |= 0x04; //启动T3

}

$$TMR3RL = 65535 - \frac{sysclk}{12 \times 10^6}$$

//T3赋初值, 也可以采用8位SFR方式

//立即重载

//开T3中断

自动重装初值.

再过一个 clk, $TMR3 = -counts$;



3. T3应用举例

`void Timer3_ISR (void) interrupt 14` // 定时器3溢出.

```
{
    static int count;
    static int i = 9, j = 0;
    static int led = 0xFF;           //P3口LED灯的初始状态
    TMR3CN &= ~(0x80);              //清TF3
    count++;
    if(count==5)                     //T3中断5次更新一次LED灯状态
    {
        count=0;
        P3=led;
        P2=p2led[j];                 //查表
        led=led << 1;
        i--;      j++;
        if(j==8) j=0;                 //P2口LED灯循环一个周期
        if(i==0) {i=9; led=0xFF;}     //P3口LED灯循环一个周期
    }
}
```

